

CHEAT SHEETS

A Complete Reference for macOS Terminal & Shell Scripting

[Navigation](#)[Files & Directories](#)[Wildcards & grep](#)[Redirection & Pipes](#)[Permissions](#)[Environment](#)[Variables & Scripting](#)[Control Flow](#)[Functions & Arrays](#)[Processes](#)[Text Processing](#)[macOS Tools](#)

12 REFERENCE SHEETS

CONTENTS

THE TERMINAL

- 01 Introduction & Navigation
- 02 Files & Directories
- 03 Viewing, Wildcards & Patterns
- 04 Redirection & Piping
- 05 Permissions & Ownership
- 06 Environment & Configuration

SCRIPTING

- 07 Script Basics & Variables
- 08 Operators & Control Flow
- 09 Functions & Arrays
- 10 Processes & Job Control
- 11 Text Processing
- 12 Script Patterns & macOS Tools

Sheet 01 Introduction & Navigation bash · zsh · pwd · cd · ls · history · man · keyboard shortcuts	Sheet 02 Files & Directories touch · cp · mv · rm · mkdir · find · stat · paths · symlinks	Sheet 03 Viewing, Wildcards & Patterns cat · less · head · tail · grep · wildcards · globbing · regex
Sheet 04 Redirection & Piping stdin · stdout · stderr · > · · tee · xargs · here-docs	Sheet 05 Permissions & Ownership chmod · chown · rwx · octal · umask · sudo · ACLs · executable	Sheet 06 Environment & Configuration \$PATH · export · alias · .zshrc · source · env · printenv · which
Sheet 07 Script Basics & Variables shebang · variables · strings · quoting · echo · read · \$!...\$@ · \$?	Sheet 08 Operators & Control Flow arithmetic · comparison · file tests · [[]] · if · case · for · while	Sheet 09 Functions & Arrays functions · arguments · local · return · arrays · declare -A · IFS
Sheet 10 Processes & Job Control ps · kill · & · jobs · fg · bg · trap · nohup · timeout · signals	Sheet 11 Text Processing sed · awk · cut · sort · uniq · tr · diff · column · one-liners	Sheet 12 Script Patterns & macOS Tools set -euo pipefail · getopts · logging · pbcopy · open · defaults · brew

WHAT IS THE SHELL?

The **shell** is a command-line interpreter that lets you interact with macOS directly. macOS ships with **Zsh** as the default shell (since macOS Catalina 10.15), but **Bash** is also available at `/bin/bash`. Scripts written for bash run with `#!/bin/bash` or the portable `#!/usr/bin/env bash` shebang. Both shells share the vast majority of everyday commands — differences are noted where they matter. The Terminal app (Terminal.app or iTerm2) is the GUI wrapper around whichever shell is active.

BASH VS ZSH

- › `echo $SHELL` — shows your current shell
- › `bash --version` — macOS ships Bash 3.2 (GPL2); install Bash 5 via Homebrew
- › Zsh is default on macOS 10.15+; config lives in `~/.zshrc`
- › Bash config lives in `~/.bash_profile` (login) or `~/.bashrc` (interactive)
- › `chsh -s /bin/zsh` — change default shell to zsh
- › `chsh -s /opt/homebrew/bin/bash` — change to Homebrew bash

NOTES

- `pwd` — always shows the full absolute path; never leaves you guessing
- `cd -` — toggles between last two dirs; great for back-and-forth work
- `ls -la` — the most common combo; shows all files with full detail
- `ls -lh` — pairs with `-l`; makes file sizes human-readable
- `ls -lt` — sort newest-first; useful for finding recent files
- `open .` — macOS-specific; bridges terminal and Finder

NAVIGATION COMMANDS

```
# — Where am I? —————
pwd                # /Users/alice (print working directory)
echo $HOME         # /Users/alice

# — Change directory —————
cd /usr/local/bin  # absolute path
cd Documents       # relative path
cd ..              # up one level
cd ../..           # up two levels
cd ~               # home directory
cd -               # previous directory (toggle)
cd                 # bare cd also goes home

# — List directory contents —————
ls                 # basic list
ls -l              # long format (permissions, size, date)
ls -la             # include hidden files (dot-files)
ls -lh             # human-readable sizes (KB, MB)
ls -lt             # sort by modification time
ls -R              # recursive listing
ls -G              # colorized output (macOS default alias)

# — Useful navigation shortcuts —————
open .             # open current dir in Finder
open -a "Visual Studio Code" . # open in app by name
```

GETTING HELP

COMMAND	DESCRIPTION
<code>man ls</code>	Full manual page for <code>ls</code>
<code>man -k keyword</code>	Search man pages by keyword
<code>ls --help</code>	Short usage summary (GNU tools)
<code>whatis ls</code>	One-line description of command
<code>type cd</code>	Shows if command is builtin, alias, or file
<code>which python3</code>	Full path to executable
<code>help</code>	List all shell builtins (bash)
<code>tldr ls</code>	Practical examples (install via brew)

TERMINAL KEYBOARD SHORTCUTS

SHORTCUT	ACTION
<code>Tab</code>	Autocomplete command or path
<code>Tab Tab</code>	Show all completions
<code>↑ / ↓</code>	Scroll command history
<code>Ctrl+R</code>	Reverse search history (type to filter)
<code>Ctrl+C</code>	Cancel current command
<code>Ctrl+D</code>	Exit shell / end of input
<code>Ctrl+L</code>	Clear screen (same as <code>clear</code>)
<code>Ctrl+A / E</code>	Jump to start / end of line
<code>Ctrl+W</code>	Delete word before cursor
<code>Ctrl+U</code>	Delete entire line
<code>Cmd+K</code>	Clear scrollbar buffer (macOS Terminal)
<code>Cmd+T</code>	New terminal tab (macOS Terminal)

COMMAND HISTORY

COMMAND / SYNTAX	DESCRIPTION	EXAMPLE
<code>history</code>	List recent commands with numbers	<code>history 20</code> — last 20
<code>!!</code>	Repeat last command	<code>sudo !!</code> — re-run as sudo
<code>!n</code>	Run command number n from history	<code>!42</code>
<code>!string</code>	Run last command starting with string	<code>!git</code>
<code>!\$</code>	Last argument of previous command	<code>mkdir foo && cd !\$</code>
<code>Ctrl+R</code>	Interactive reverse history search	Type to filter, Enter to run
<code>history -c</code>	Clear history (bash)	<code>fc -p</code> in zsh

COMMON MISTAKES

- › `cd` to a path with spaces requires quotes: `cd "My Folder"` or escape: `cd My\ Folder`
- › Forgetting `ls -a` and missing hidden dot-files like `.gitignore` or `.env`
- › `man` pages use `q` to quit and `/term` to search — new users often get stuck
- › Assuming bash and zsh behave identically — array syntax and some expansions differ

TIPS

- › Press `Tab` aggressively — the shell completes paths, commands, and flags
- › Use `Ctrl+R` to search history instead of pressing `↑` dozens of times
- › Install `tldr` via Homebrew for quick, practical command examples alongside `man`
- › `type command` tells you exactly what the shell will execute — alias, function, or binary

SUMMARY

- › `pwd / cd / ls` — the three core navigation commands
- › macOS default is Zsh; Bash 5 available via Homebrew
- › `Tab`, `Ctrl+R`, `!!` — essential productivity shortcuts
- › `man` and `tldr` — two levels of help, built-in and practical

FILE SYSTEM BASICS

The macOS file system is hierarchical, rooted at /. Every file and directory has an **absolute path** (starts with /, e.g. /Users/alice/Documents) or a **relative path** (relative to your current directory, e.g. ../config). Special shorthand: ~ is your home directory, . is the current directory, and .. is the parent. macOS is case-insensitive by default but case-preserving — File.txt and file.txt are the same file on most macOS volumes.

PATH QUICK REFERENCE

- › / — root of the entire file system
- › ~ — expands to /Users/username
- › . — current directory
- › .. — parent directory
- › - — previous directory (with cd)
- › Spaces in paths: use quotes "My Dir" or escape My\ Dir

NOTES

- touch — creates file if missing; also updates access/modify time on existing files
- mkdir -p — the -p flag is essential; without it, parent dirs must already exist
- cp -r — required for directories; plain cp fails on dirs
- rm — bypasses the Trash entirely; there is no undo. Use -i to be safe
- rm -rf — dangerous but common; double-check the path before running
- stat — shows inode, permissions, size, timestamps in one view
- ln -s — symlinks point to a path; they break if the target moves

FILE & DIRECTORY COMMANDS

```
# — Create —
touch file.txt           # create empty file (or update timestamp)
touch file1.txt file2.txt # create multiple files
mkdir mydir              # create directory
mkdir -p a/b/c           # create nested dirs (no error if exists)

# — Copy —
cp file.txt backup.txt   # copy file
cp -r src/ dest/         # copy directory recursively
cp -i file.txt dest/     # prompt before overwrite
cp -p file.txt dest/     # preserve permissions and timestamps

# — Move / Rename —
mv file.txt newname.txt  # rename file
mv file.txt ~/Documents/ # move to directory
mv -i file.txt dest/     # prompt before overwrite

# — Delete —
rm file.txt              # delete file (no Trash — permanent!)
rm -i file.txt           # prompt before delete
rm -r mydir/             # delete directory recursively
rm -rf mydir/            # force-delete, no prompts (be careful!)

# — Inspect —
stat file.txt            # full file metadata
file file.txt            # detect file type by content
du -sh mydir/            # disk usage, human-readable
du -sh *                 # size of every item in current dir

# — Symlinks —
ln -s /path/to/original link_name # create symbolic link
ls -la                   # symlinks show as link -> target
readlink link_name       # show link target
```

FIND — SEARCH FOR FILES

COMMAND	DESCRIPTION
find . -name "*.txt"	Find by name (case-sensitive)
find . -iname "*.txt"	Find by name (case-insensitive)
find . -type f	Files only (not directories)
find . -type d	Directories only
find . -size +1M	Files larger than 1 megabyte
find . -mtime -7	Modified in last 7 days
find . -name "*.log" -delete	Find and delete matches
find . -name "*.py" -exec wc -l {} \;	Run command on each result
find /usr -maxdepth 2 -name "*.conf"	Limit search depth

COMMON MISTAKES

- › rm -rf / or rm -rf ~ — catastrophic; always double-check paths before running
- › Using cp without -r on a directory silently fails or errors
- › Forgetting mkdir -p and getting "No such file or directory" for missing parent dirs
- › Trailing slash matters: cp src dest/ vs cp src dest behave differently

TIPS

- › Alias rm to trash (brew install trash) to send files to macOS Trash instead of permanent deletion
- › Use find . -name "*.log" -delete instead of rm for bulk deletes — safer path matching
- › cp -rp preserves ownership/timestamps — essential when duplicating config dirs
- › Use mkdir -p dist/{css,js,img} with brace expansion to build entire folder trees at once

USEFUL FILE COMMANDS

COMMAND	DESCRIPTION
ls -lh	Human-readable file sizes
ls -lt	Sort by modification time
ls -ls	Sort by file size
du -sh *	Size of every item here
df -h	Disk free space (all volumes)
file image.png	Detect file type from content
basename /a/b/c.txt	Returns c.txt
dirname /a/b/c.txt	Returns /a/b
realpath ./file.txt	Resolve to absolute path

SUMMARY

- › touch · mkdir -p — create files and nested dirs
- › cp -r · mv — copy and move; always -r for dirs
- › rm is permanent — use -i flag or trash CLI for safety
- › find — powerful recursive search with filters for name, type, size, date
- › stat · du · df — inspect file metadata and disk usage

NOTES

- **cat** — best for short files; for long files use **less**
- **less** — navigate with arrow keys, **Space** =page down, **b** =page up, **q** =quit
- **tail -f** — indispensable for monitoring log files in real time
- **grep -r** — searches all files recursively; combine with **-l** for file list
- **grep -E** — enables extended regex; same as using **egrep**
- **grep -A/-B/-C** — show context lines around matches

WILDCARDS & GLOBBING

PATTERN	MATCHES	EXAMPLE
*	Any string (including empty)	ls *.txt
?	Any single character	ls file?.txt
[abc]	Any one of: a, b, or c	ls file[123].txt
[a-z]	Any character in range	ls [a-z]*.sh
[^abc]	Any character NOT in set	ls [^0-9]*
**	Recursive match (globstar)	ls **/*.js
{a,b,c}	Brace expansion — literal list	cp {a,b}.txt dest/
{1..5}	Brace expansion — sequence	touch file{1..5}.txt

BASIC REGEX (GREP)

PATTERN	MATCHES
.	Any single character
^	Start of line
\$	End of line
*	Zero or more of preceding
+	One or more (ERE / grep -E)
?	Zero or one (ERE / grep -E)
[abc]	Character class
[^abc]	Negated character class
\b	Word boundary
(a b)	Alternation (ERE / grep -E)
\d	Digit (grep -P for Perl regex)
{2,4}	Repeat 2–4 times (ERE)

PRACTICAL GREP & PATTERN EXAMPLES

COMMAND	WHAT IT DOES
grep -rn "TODO" . --include="*.py"	Find TODOs in Python files only, show line numbers
grep -E "^ [0-9]{4}-" file.txt	Lines starting with a 4-digit year
grep -v "^#" config.txt	Strip comment lines (starting with #)
grep -o "[0-9]\+" file.txt	Extract only the matched numbers (-o = only)
ls *. {jpg,png,gif}	List all image files by extension
cp file.txt{, .bak}	Quick backup: copies to file.txt.bak
echo {a..z}	Prints: a b c d ... z
ls **/*.test.js	All test files in all subdirs (requires globstar on)

COMMON MISTAKES

- › Quoting glob patterns accidentally: **grep "*.txt"** searches for literal ***.txt**; the shell expands globs, not **grep**
- › ****** globbing requires **shopt -s globstar** in bash or **setopt globstar** in zsh — not on by default in bash
- › Forgetting **-E** for extended regex — **+**, **|**, and **()** need it in basic **grep**

TIPS

- › Use **grep --color=auto** (often aliased by default) to highlight matches visually
- › **tail -f** combined with **grep --line-buffered** filters a live log stream
- › The brace trick **file.txt{,.bak}** expands to two args — perfect for quick backups
- › Use **less +F** instead of **tail -f** — it lets you toggle follow mode with **Ctrl+C** then scroll, then **F** to resume

SUMMARY

- › **cat · less · head · tail** — four levels of file viewing
- › **grep** — search file content; **-r**, **-i**, **-n**, **-E** are the key flags
- › *** ? []** — shell globs; **{a,b}** **{1..n}** — brace expansion
- › Regex in **grep**: basic by default; use **-E** for extended, **-P** for Perl

VIEWING FILE CONTENTS

```
# — Display entire file
cat file.txt                # print file to terminal
cat -n file.txt             # with line numbers
cat file1.txt file2.txt     # concatenate multiple files

# — Paged viewing (less)
less file.txt               # open pager (q=quit, /=search, n=next)
less +G file.txt            # open at end of file
less +F file.txt            # follow mode (like tail -f)

# — Head and tail
head file.txt               # first 10 lines (default)
head -n 20 file.txt         # first 20 lines
tail file.txt               # last 10 lines
tail -n 20 file.txt         # last 20 lines
tail -f logfile.log         # follow file (live updates – great for logs)
tail -f -n 50 app.log       # follow last 50 lines

# — Word/line counts
wc file.txt                 # lines words bytes
wc -l file.txt              # line count only
wc -w file.txt              # word count only
wc -c file.txt              # byte count only

# — grep – search content
grep "error" file.txt       # lines containing "error"
grep -i "error" file.txt    # case-insensitive
grep -n "error" file.txt    # show line numbers
grep -r "TODO" ./src       # search recursively
grep -l "error" *.log       # filenames only (not lines)
grep -v "debug" file.txt    # invert – lines NOT matching
grep -c "error" file.txt    # count matching lines
grep -E "err|warn" file.txt # extended regex (alternation)
grep -A 2 -B 2 "error" file # 2 lines after and before match
```

STANDARD STREAMS

Every process has three standard streams: **stdin** (0 — input), **stdout** (1 — normal output), and **stderr** (2 — error output). By default all three connect to the terminal. **Redirection** detaches a stream and points it at a file. **Piping** (|) connects the stdout of one command directly to the stdin of the next, creating powerful data-processing chains without temporary files.

STREAM NUMBERS

- 0 — stdin (keyboard by default)
- 1 — stdout (terminal by default)
- 2 — stderr (terminal by default)
- 2>&1 — redirect stderr into stdout stream
- /dev/null — discard anything sent to it
- Order matters: `cmd > f 2>&1` not `cmd 2>&1 > f`

NOTES

- > truncates first, then writes — existing content is gone immediately
- >> — safe append; never overwrites existing content
- 2>&1 — must come after > file; order determines what stream 2 joins
- tee — invaluable in pipelines where you want to log and continue processing
- xargs — converts stdin lines into command arguments; use -n1 for one arg at a time
- << EOF — here-doc; the delimiter (EOF) can be any word; indent with <<-
- \$(cmd) — preferred over backticks; nestable and readable
- <(cmd) — process substitution; treats command output as a temporary file

REDIRECTION QUICK REFERENCE

SYNTAX	MEANING
cmd > file	stdout → file (overwrite)
cmd >> file	stdout → file (append)
cmd < file	file → stdin
cmd 2> file	stderr → file
cmd 2>&1	stderr → stdout
cmd &> file	stdout + stderr → file
cmd > /dev/null	Discard stdout
cmd >/dev/null 2>&1	Discard all output
cmd << EOF	Here-doc (multi-line input)
cmd <<< "str"	Here-string (single-line input)

COMMON PIPE PATTERNS

PATTERN	PURPOSE
cmd grep pattern	Filter output by pattern
cmd sort uniq	Sort and deduplicate
cmd sort uniq -c	Count occurrences
cmd head -20	Show first 20 lines
cmd tail -20	Show last 20 lines
cmd wc -l	Count output lines
cmd tee file.txt	Save and display
cmd less	Page through output
cmd xargs cmd2	Use output as args

COMMON MISTAKES

- cmd 2>&1 > file — wrong order; stderr redirect happens before stdout changes; use `cmd > file 2>&1`
- Using > when you meant >> — silently destroys file contents before writing
- Piping to `xargs rm` without testing with `echo` first — run `| xargs echo rm` to preview

TIPS

- Preview any destructive `xargs` command by replacing the action with `echo` first
- Use `| tee /dev/stderr | next-cmd` to debug mid-pipeline without breaking the chain
- `cmd &> /dev/null` is bash shorthand for `> /dev/null 2>&1` — cleaner for silencing noisy commands
- Here-docs with `<<-EOF` (dash) strip leading tabs — useful for indented scripts

REDIRECTION & PIPE EXAMPLES

```
# — Output redirection —————
echo "hello" > file.txt           # write stdout to file (overwrite)
echo "world" >> file.txt          # append stdout to file
ls > filelist.txt 2>&1            # stdout + stderr → file
ls 2> errors.txt                 # stderr only → file
ls > out.txt 2> err.txt          # stdout and stderr to separate files
ls > /dev/null                   # discard stdout entirely
ls 2>/dev/null                   # discard errors only

# — Input redirection —————
sort < names.txt                 # feed file as stdin to sort
wc -l < file.txt                 # count lines from file

# — Pipes —————
ls -la | less                    # page long output
cat file.txt | sort | uniq       # chain: sort then deduplicate
ps aux | grep "python"          # filter process list
history | grep "git"             # search command history
ls *.log | wc -l                 # count log files

# — tee — split output —————
ls | tee filelist.txt            # print AND save to file
ls | tee -a filelist.txt         # print AND append to file
cmd | tee file.txt | grep x      # branch: save all, filter to terminal

# — xargs — pipe to arguments —————
find . -name "*.tmp" | xargs rm   # delete found files
find . -name "*.txt" | xargs wc -l # count lines in all txt files
echo "a b c" | xargs mkdir        # create dirs a, b, c
cat urls.txt | xargs -n1 curl -O  # download each URL

# — Here-doc (multi-line stdin) —————
cat << EOF
Line one
Line two
EOF

# — Here-string (single string as stdin) —————
wc -w <<< "count these words"    # 3
base64 <<< "hello"               # encodes string directly

# — Command substitution —————
now=$(date +%Y-%m-%d)           # capture command output as variable
files=$(ls *.txt)                # store file list
echo "Today: $now"

# — Process substitution —————
diff <(sort file1.txt) <(sort file2.txt) # diff sorted versions
```

SUMMARY

- > overwrites · >> appends · < reads from file
- 2>&1 — merge stderr into stdout; order after > matters
- | chains commands; tee splits; xargs converts lines to args
- \$(cmd) captures output · <<-EOF feeds multi-line input

UNIX PERMISSIONS MODEL

Every file has an **owner** (user), a **group**, and **permission bits** for three audiences: owner (**u**), group (**g**), and others (**o**). Each audience gets three bits: read (4), write (2), execute (1). Run `ls -l` to see them: `-rwxr-xr--` means owner=rwx, group=r-x, others=r--. macOS also supports extended **ACLs** (Access Control Lists) visible as `+` on `ls -l` and inspected with `ls -le`.

CHMOD, CHOWN & SUDO

```
# — chmod — change permissions —————
chmod 755 script.sh      # rwxr-xr-x (octal)
chmod 644 file.txt       # rw-r--r-- (octal)
chmod 600 secret.key     # rw----- (owner only)
chmod 777 file           # rwxrwxrwx (avoid — grants all)
chmod +x script.sh       # add execute for all
chmod -x script.sh       # remove execute for all
chmod u+x script.sh      # add execute for owner only
chmod g-w file.txt       # remove write from group
chmod o= file.txt        # clear all permissions for others
chmod -R 755 mydir/      # recursive chmod on directory

# — chown — change owner / group —————
chown alice file.txt     # change owner to alice
chown alice:staff file.txt # change owner and group
chown :staff file.txt    # change group only
chown -R alice:staff mydir/ # recursive ownership change
whoami                  # print current username
id                       # uid, gid, and all groups

# — sudo & su —————
sudo command             # run command as root
sudo -i                  # start interactive root shell
sudo -u alice command    # run as different user
su - alice                # switch to alice (requires password)

# — umask — default permissions for new files —————
umask                    # show current umask (e.g. 022)
umask 022                # dirs=755, files=644 (common default)
umask 077                # dirs=700, files=600 (private)

# — macOS ACLs —————
ls -le file.txt          # show ACL entries
chmod +a "alice allow read,write" file.txt # add ACL entry
chmod -a "alice allow read,write" file.txt # remove ACL entry
```

COMMON MISTAKES

- Setting `777` to "fix" permission errors — always find the actual minimal permission needed instead
- Running scripts as `sudo` when the issue is just a missing `chmod +x`
- Forgetting that directories need `x` permission to be entered — `r` alone only lets you list, not `cd`
- `chmod -R 777 /` or similar recursive mistakes on system directories — catastrophic

TIPS

- Use `sudo !!` immediately after a "permission denied" error to retry with root
- Always verify new scripts with `ls -l script.sh` before running — confirm the `x` bit is set
- For SSH: `~/.ssh` must be `700`, key files `600`, `authorized_keys` `644` — SSH rejects wrong perms
- Use `stat -f "%Sp %Su %Sg" file` on macOS for a scriptable permission/owner summary

READING LS -L OUTPUT

- `-rwxr-xr--` — file type + 9 permission bits
- Position 1: `-` =file, `d` =dir, `l` =symlink
- Bits 2–4: owner (u), bits 5–7: group (g), bits 8–10: others (o)
- `r` =4, `w` =2, `x` =1 — add for octal value
- `755` = `rwxr-xr-x` (typical dir), `644` = `rw-r--r--` (typical file)
- `@` after perms = extended attributes; `+` = ACL

NOTES

- `chmod +x` — the most common use; makes a script executable without touching other bits
- `chmod -R` — recursive; apply carefully — setting wrong perms on `/` breaks the system
- `600` — for private keys and secrets; readable only by owner
- `chown -R` — often needed after copying files or running commands as root
- `sudo !!` — re-run the last command with sudo (very common workflow)
- `umask` — the value is subtracted from 666 (files) or 777 (dirs) to set defaults
- `ls -le` — macOS-specific; shows extended ACLs below each file entry

COMMON OCTAL VALUES

OCTAL	SYMBOLIC	TYPICAL USE
777	rwxrwxrwx	Full access all — avoid
755	rwxr-xr-x	Directories, public scripts
750	rwxr-x---	Dir, group readable only
700	rwx-----	Dir, owner only
664	rw-rw-r--	Shared team files
644	rw-r--r--	Normal files (default)
640	rw-r-----	Config files (no others)
600	rw-----	SSH keys, secrets
444	r--r--r--	Read-only all

SYMBOLIC CHMOD SYNTAX

COMMAND	EFFECT
<code>chmod u+x</code>	Add execute for owner
<code>chmod g-w</code>	Remove write from group
<code>chmod o=r</code>	Set others to read only
<code>chmod a+r</code>	Add read for all (a=ugo)
<code>chmod ug+rw</code>	Add read+write for user+group
<code>chmod o-rwx</code>	Remove all perms from others
<code>chmod u=rwx,go=rx</code>	Set exactly: owner=rwx, rest=rx

SUMMARY

- Three audiences: `u` ser · `g` roup · `o` thers; three bits each: `r` =4 `w` =2 `x` =1
- `chmod 755` (dirs) · `644` (files) · `600` (secrets) — the three defaults
- `chmod +x` — makes scripts executable; required before running a script
- `sudo !!` — the fastest way to retry with elevated privileges

THE SHELL ENVIRONMENT

The shell maintains a set of **environment variables** — key-value pairs inherited by every child process. The most important is `$PATH`, a colon-separated list of directories the shell searches when you type a command name. Configuration files (`~/.zshrc` for Zsh, `~/.bash_profile` for Bash) are sourced at startup, making aliases, variables, and functions available in every session. Use `source` (or `.`) to reload them without restarting the terminal.

CONFIG FILE LOAD ORDER

- › **Zsh login shell:** `/etc/zprofile` → `~/.zprofile` → `~/.zshrc`
- › **Bash login shell:** `/etc/profile` → `~/.bash_profile`
- › **Bash interactive:** `~/.bashrc` (non-login)
- › macOS Terminal opens a login shell by default
- › Changes take effect next session or after `source ~/.zshrc`
- › `~/.zshenv` — sourced for every zsh, including scripts

ENVIRONMENT COMMANDS

```
# — View environment —————
env                # all environment variables
printenv           # same as env
printenv PATH      # value of one variable
echo $HOME         # /Users/alice
echo $SHELL        # /bin/zsh
echo $USER         # alice

# — Set & export variables —————
MY_VAR="hello"     # set shell variable (not exported)
export MY_VAR="hello" # export to child processes
export PATH="$PATH:/usr/local/bin" # append to PATH
export EDITOR="vim"  # set default editor

# — $PATH management —————
echo $PATH         # show current PATH (colon-separated)
which python3      # /usr/bin/python3 — where it resolves
which -a python3   # all matches in PATH order
type python3       # python3 is /usr/bin/python3

# — Aliases —————
alias ll="ls -la"   # define alias
alias gs="git status"
alias reload="source ~/.zshrc"
alias ..="cd .."
alias ...="cd ../../"
unalias ll          # remove alias
alias              # list all aliases

# — source — reload config files —————
source ~/.zshrc     # reload zsh config
source ~/.bash_profile # reload bash config
. ~/.zshrc         # dot is shorthand for source

# — Common useful variables —————
echo $HOME         # /Users/alice
echo $PWD          # current directory (same as pwd)
echo $OLDPWD       # previous directory
echo $?           # exit code of last command
echo $$           # current shell PID
echo $RANDOM       # random integer 0–32767
```

COMMON MISTAKES

- › Setting `PATH="/new/path"` instead of `PATH="/new/path:$PATH"` — replaces `PATH` entirely, breaking all commands
- › Forgetting `export` — variable set without it won't be visible to scripts or child processes
- › Editing `~/.zshrc` but forgetting to `source` it — changes won't apply until you do
- › Defining aliases with spaces around `=`: `alias ll = "ls"` fails; must be `alias ll="ls"`

NOTES

- `export` — without it, a variable is only visible in the current shell, not scripts or subshells
- `$PATH` — order matters; earlier dirs win; always append with `"$PATH:/new"` not replace
- `which -a` — reveals if a command is shadowed by another earlier in `PATH`
- `alias` — only lives for the session; add to `~/.zshrc` to persist
- `source` vs running a script — `source` runs in current shell; `./script.sh` runs in a subshell
- `$?` — check after any command; 0=success, non-zero=error

COMMON ENVIRONMENT VARIABLES

VARIABLE	DESCRIPTION
<code>\$PATH</code>	Colon-separated command search directories
<code>\$HOME</code>	Current user's home directory
<code>\$USER</code>	Current username
<code>\$SHELL</code>	Path to the current shell binary
<code>\$PWD</code>	Current working directory
<code>\$EDITOR</code>	Default text editor (vim, nano, code...)
<code>\$TERM</code>	Terminal type (xterm-256color etc.)
<code>\$LANG</code>	Locale setting (en_US.UTF-8)
<code>\$TMPDIR</code>	macOS temp directory path
<code>\$?</code>	Exit code of last command (0=success)
<code>\$\$</code>	PID of the current shell

~/.ZSHRC STARTER PATTERNS

ENTRY	PURPOSE
<code>export PATH="\$HOME/bin:\$PATH"</code>	Add <code>~/bin</code> to <code>PATH</code> first
<code>export EDITOR="code --wait"</code>	VS Code as default editor
<code>alias ll="ls -laG"</code>	Long list with color
<code>alias reload="source ~/.zshrc"</code>	Quick config reload
<code>alias python="python3"</code>	Always use python3
<code>export HISTSIZE=10000</code>	Keep 10k history entries
<code>setopt HIST_IGNORE_DUPS</code>	Don't store duplicate history
<code>autoload -U compinit && compinit</code>	Enable tab completion

TIPS

- › Keep a `~/bin/` directory for personal scripts and add it first in `$PATH` — it'll always take precedence
- › Use `which -a` command to debug "wrong version" issues — reveals every match in `PATH` order
- › Add `alias reload="source ~/.zshrc"` to your config — you'll use it constantly
- › Use `command -v tool` in scripts instead of `which` — it's POSIX portable and returns an exit code

SUMMARY

- › `$PATH` — the most important env var; always append, never replace
- › `export VAR=value` — makes variables available to child processes
- › `alias` — shortcuts; add to `~/.zshrc` to persist across sessions
- › `source ~/.zshrc` — reload config without restarting terminal

WRITING SHELL SCRIPTS

A shell script is a plain text file containing shell commands. The first line — the **shebang** (`#!/usr/bin/env bash`) — tells the OS which interpreter to use. Make it executable with `chmod +x script.sh`, then run it with `./script.sh`. Variables are untyped by default; everything is a string unless used in an arithmetic context. Variable names are case-sensitive; by convention, local variables use lowercase and environment variables uppercase.

SCRIPT SYNTAX

```
#!/usr/bin/env bash
# — Script header —————
# Good practice: set -euo pipefail (see Sheet 12)

# — Variables —————
name="Alice"           # string variable
count=42               # integer (still stored as string)
greeting="Hello, $name!" # interpolation in double quotes
literal='Hello, $name!'  # no interpolation in single quotes
echo $greeting          # Hello, Alice!
echo $literal            # Hello, $name!

# — Braces & default values —————
echo "${name}s"         # Alices — brace required before s
echo "${unset_var:-default}" # use default if var is unset or empty
echo "${count:=10}"     # assign 10 if count is unset
echo "${name:+exists}"  # "exists" only if name is set
echo "${#name}"         # 5 (string length)

# — echo & printf —————
echo "Hello World"      # print with newline
echo -n "no newline"    # suppress trailing newline
echo -e "tab:\there"    # interpret escape sequences
printf "%-10s %d\n" "item" 5 # formatted output (like C printf)
printf "%05d\n" 42        # 00042 — zero-padded

# — read — get user input —————
read name               # read from stdin into $name
read -p "Enter name: " name # with prompt
read -s -p "Password: " pass # silent (no echo)
read -t 5 name          # timeout after 5 seconds
read -r line            # raw mode — don't interpret backslashes
read -a arr             # read words into array

# — Script arguments —————
# ./myscript.sh Alice 42
echo $0                 # script name: ./myscript.sh
echo $1                 # first arg: Alice
echo $2                 # second arg: 42
echo $@                 # all args: Alice 42
echo $*                 # all args as single string
echo $#                 # number of args: 2
echo $?                 # exit code of last command
shift                  # shift args left ($2 becomes $1)
```

COMMON MISTAKES

- Spaces around `=`: `x = 5` fails — shell interprets it as a command named `x`
- Using `$*` instead of `"$@"` — arguments with spaces get split incorrectly
- Forgetting to quote variables: `rm $file` breaks on filenames with spaces; use `rm "$file"`
- Checking `?` too late — each command overwrites it; capture immediately after the command

TIPS

- Always quote variable references in scripts: `"$var"` — prevents word-splitting and glob expansion
- Use `${var:?Error message}` at the top of scripts to fail fast on missing required variables
- `printf` is more portable than `echo` for scripts that need to run on multiple systems
- Use `local var=value` inside functions to avoid polluting the global namespace

VARIABLE RULES

- No spaces around `=`: `name="alice"` ✓ — `name = "alice"` ✗
- Reference with `$name` or `${name}` — braces required before text
- Unset variable expands to empty string (no error by default)
- `readonly VAR=value` — makes variable immutable
- `unset VAR` — delete a variable
- Single quotes: literal, no expansion. Double quotes: expands `$` and ```

NOTES

- `#!/usr/bin/env bash` — preferred over `#!/bin/bash`; finds bash in PATH, more portable
- `${var:-default}` — the safe expansion family; avoids "unbound variable" errors
- `${#name}` — string length operator; works for any variable
- `printf` — more reliable than `echo` for formatted or multi-line output in scripts
- `read -r` — always use `-r` in scripts; prevents backslash interpretation surprises
- `$@` vs `$*` — prefer `"$@"` (quoted); preserves args with spaces as separate items
- `?` — check immediately after the command; the next command will overwrite it

SPECIAL VARIABLES

VARIABLE	DESCRIPTION
\$0	Script name / path
\$1 ... \$9	Positional arguments
\${10}	Arg 10+ requires braces
\$@	All arguments (as separate words)
\$*	All arguments (as one word)
\$#	Number of arguments
?	Exit code of last command
\$\$	PID of current shell
!	PID of last background process
\$-	Active shell option flags
_	Last argument of previous command

PARAMETER EXPANSION

SYNTAX	MEANING
\${var:-default}	Use default if unset or empty
\${var:=default}	Assign default if unset
\${var+=value}	Use value only if var is set
\${var:?message}	Error with message if unset
\${#var}	Length of string
\${var:2:5}	Substring: offset 2, length 5
\${var#prefix}	Remove shortest prefix match
\${var##prefix}	Remove longest prefix match
\${var#suffix}	Remove shortest suffix match
\${var%%suffix}	Remove longest suffix match
\${var/old/new}	Replace first occurrence
\${var//old/new}	Replace all occurrences
\${var^^}	Uppercase all (bash 4+)
\${var,,}	Lowercase all (bash 4+)

SUMMARY

- Shebang `#!/usr/bin/env bash` + `chmod +x` = runnable script
- No spaces around `=`; always quote `"$variables"` in scripts
- `$!..$@` — script arguments; `$#` = count; `?` = last exit code
- `${var:-default}` family — safe expansions for robust scripts

OPERATORS & CONTROL FLOW

```
# — Arithmetic —————
echo $((5 + 3))           # 8
echo $((10 % 3))          # 1 (modulo)
echo $((2 ** 8))          # 256
(( count++ ))             # increment
(( total = a + b ))       # assign result
x=$(( width * height ))  # store result

# — Conditionals: if / elif / else —————
if [[ $name == "Alice" ]]; then
    echo "Hello Alice"
elif [[ $age -gt 18 ]]; then
    echo "Adult"
else
    echo "Other"
fi

# — One-liners (short-circuit) —————
[[ -f file.txt ]] && echo "exists"    # run if true
[[ -z $var ]] && echo "empty"         # var is empty
mkdir -p dir || echo "failed"       # run if false

# — case statement —————
case $fruit in
    apple) echo "Apple" ;;
    banana|mango) echo "Tropical" ;;
    *)      echo "Other" ;;
esac

# — for loops —————
for i in 1 2 3 4 5; do echo $i; done
for i in {1..10}; do echo $i; done      # sequence
for i in {0..20..2}; do echo $i; done   # step by 2
for f in *.txt; do echo "$f"; done      # glob
for i in $(seq 1 5); do echo $i; done    # seq command
for (( i=0; i<5; i++ )); do echo $i; done # C-style

# — while / until —————
count=0
while [[ $count -lt 5 ]]; do
    echo $count
    (( count++ ))
done

until [[ $count -ge 5 ]]; do    # until = while NOT
    echo $count; (( count++ ))
done

# — break / continue —————
for i in {1..10}; do
    [[ $i -eq 5 ]] && break      # exit loop
    [[ $i -eq 3 ]] && continue   # skip iteration
    echo $i
done
```

COMMON MISTAKES

- Using `[]` instead of `[[ ]]` — single brackets don't support `&&`, `||`, or regex; use double always in bash
- Mixing numeric and string operators: `[[ "10" > "9" ]]` is false (string sort) — use `-gt` for numbers
- Missing `;` before `then` / `do` on the same line: `if [[ ... ]]` then fails; needs `]] ; then`
- Not quoting loop variables: `for f in *.txt` is fine but use `"$f"` inside the loop body

TIPS

- Use `&&` / `||` for simple one-liners instead of full `if` blocks — `[[ -d dist ]] || mkdir dist`
- Regex with `=~` in `[[ ]]`: don't quote the pattern — `[[ $str =~ ^[0-9]+$ ]]`
- Use `case` for 3+ branches on the same variable — far more readable than `elif` chains
- `for f in *.txt` safely handles no matches with `nullglob` option (`shopt -s nullglob`)

NOTES

- `(( ))` — arithmetic context; no `$` needed on variable names inside
- `[[ ]]` — preferred over `[]`; supports `&&`, `||`, regex, and no word splitting
- `&&` / `||` — short-circuit: right side runs only if left side succeeds/fails
- `case` — cleaner than long `if/elif` chains; `|` separates alternatives per pattern
- `for f in *.txt` — glob expands before loop runs; always quote `"$f"` in the body
- `until` — runs while condition is false; inverse of `while`
- `break 2` — exits two levels of nested loops

COMPARISON OPERATORS

OPERATOR	NUMERIC	STRING ([[]])
equal	<code>-eq</code>	<code>==</code>
not equal	<code>-ne</code>	<code>!=</code>
less than	<code>-lt</code>	<code><</code>
less or equal	<code>-le</code>	<code>-</code>
greater than	<code>-gt</code>	<code>></code>
greater or equal	<code>-ge</code>	<code>-</code>
regex match	<code>-</code>	<code>=~</code>
logical AND	<code>-a</code>	<code>&&</code>
logical OR	<code>-o</code>	<code> </code>
logical NOT	<code>!</code>	<code>!</code>

FILE TEST OPERATORS

TEST	TRUE IF...
<code>-e file</code>	File exists (any type)
<code>-f file</code>	Regular file exists
<code>-d file</code>	Directory exists
<code>-L file</code>	Symlink exists
<code>-r file</code>	File is readable
<code>-w file</code>	File is writable
<code>-x file</code>	File is executable
<code>-s file</code>	File is not empty (size > 0)
<code>-z str</code>	String is empty
<code>-n str</code>	String is not empty
<code>f1 -nt f2</code>	f1 is newer than f2
<code>f1 -ot f2</code>	f1 is older than f2

SUMMARY

- `[[ ]]` for conditions · `(( ))` for arithmetic · never mix them
- Numeric: `-eq -lt -gt` · String: `== != -z -n` · File: `-f -d -e -x`
- `if/elif/else/fi` · `case/esac` · `for/while/until` loops
- `break` exits loop · `continue` skips to next iteration

FUNCTIONS & ARRAYS

```
# — Define and call functions —————
greet() {
    echo "Hello, $1!"          # $1 = first arg to function
}
greet Alice                  # Hello, Alice!

# Both syntaxes work (prefer no 'function' keyword for portability)
function say_hi { echo "Hi!"; }

# — Local variables —————
counter() {
    local count=0             # local: not visible outside function
    (( count++ ))
    echo $count
}

# — return values —————
is_even() {
    (( $1 % 2 == 0 ))         # return 0 (true) if even, 1 (false) if not
}
is_even 4 && echo "even"      # even

# Capture output as return value:
get_date() { date +%Y-%m-%d; }
today=$(get_date)            # 2025-01-15

# — Indexed arrays —————
fruits=("apple" "banana" "cherry")
echo ${fruits[0]}             # apple
echo ${fruits[1]}             # banana
echo ${fruits[-1]}            # cherry (last element, bash 4.2+)
echo ${fruits[@]}             # all elements
echo ${#fruits[@]}            # 3 (length)
fruits+=(mango)               # append element
fruits[1]="blueberry"         # replace element
unset fruits[2]               # delete element (leaves gap)

# — Loop over array —————
for fruit in "${fruits[@]"; do
    echo "$fruit"
done

# — Array slices —————
echo "${fruits[@]:1:2}"        # elements 1 and 2 (offset:length)

# — Associative arrays (bash 4+) —————
declare -A person
person[name]="Alice"
person[age]=30
person[city]="Chicago"
echo ${person[name]}          # Alice
echo ${!person[@]}            # all keys: name age city
echo ${person[@]}             # all values

for key in "${!person[@]"; do
    echo "$key = ${person[$key]}"
done

# — IFS — field separator —————
IFS="," read -ra parts <<< "a,b,c"
echo ${parts[0]}              # a
echo ${parts[@]}              # a b c

# Split a string into array:
str="one:two:three"
IFS=":" read -ra arr <<< "$str"
```

COMMON MISTAKES

- Using `${arr[*]}` instead of `"${arr[@]}"` — the `*` form collapses elements with spaces into one word
- Forgetting `declare -A` before using associative arrays — silently creates a broken indexed array
- Trying to `return "a string"` from a function — `return` only accepts integers 0–255; use `echo` instead
- Modifying `IFS` globally without restoring it — breaks word-splitting in the rest of the script

TIPS

- Put reusable functions in a separate `lib.sh` file and `source` it — keeps scripts clean and testable
- Use `mapfile -t arr < file.txt` to read a file into an array (one line per element) — cleaner than `IFS` tricks
- Check if a function exists: `declare -f fn_name > /dev/null && echo "exists"`
- Always use `local` for every variable inside functions — prevents subtle bugs from global state pollution

FUNCTIONS & ARRAYS

Functions let you group commands under a name and call them like built-ins — they share the shell's environment but can use `local` to scope variables. A function's **return** value is always an exit code (0 = success); pass data back with `echo` and capture it via `$()`. **Indexed arrays** store ordered lists; **associative arrays** (`declare -A`) map arbitrary string keys to values. Both require `"${arr[@]}"` (quoted, @) to expand safely — `*` or unquoted forms break on elements with spaces.

NOTES

- `local` — essential in functions; without it, variables leak to global scope
- `return` — returns an exit code (0–255) only, not a value; use `echo + $()` for values
- `"${arr[@]}"` — always quote with `@`; each element stays as a separate word
- `${#arr[@]}` — array length; `${#arr[0]}` is length of first element string
- `declare -A` — required for associative arrays; must be declared before use
- `${!arr[@]}` — the `!` gives you keys/indices, not values
- `IFS` — save/restore if changing it: `old_IFS=$IFS; IFS=","; ...; IFS=$old_IFS`

ARRAY OPERATIONS

SYNTAX	DESCRIPTION
<code>arr=(a b c)</code>	Define indexed array
<code>\${arr[0]}</code>	First element
<code>\${arr[-1]}</code>	Last element (bash 4.2+)
<code>\${arr[@]}</code>	All elements
<code>\${#arr[@]}</code>	Number of elements
<code>arr+=(d e)</code>	Append elements
<code>unset arr[1]</code>	Delete element (leaves gap)
<code>\${arr[@]:2:3}</code>	Slice: 3 elements from index 2
<code>\${arr[@]/#/-}</code>	Prepend "-" to every element
<code>sorted=\$(printf '%s\n' "\${arr[@]}" sort)</code>	Sort array

FUNCTION PATTERNS

PATTERN	DESCRIPTION
<code>fn() { ...; }</code>	Define function (POSIX style)
<code>local var=val</code>	Scoped variable
<code>return 0 / 1</code>	Success / failure exit code
<code>result=\$(fn)</code>	Capture function output
<code>fn "\$@"</code>	Pass all script args to function
<code>declare -f fn</code>	Print function definition
<code>unset -f fn</code>	Remove function
<code>source lib.sh</code>	Import functions from file

SUMMARY

- `fn() { ...; }` — define; call like any command; uses `$1 ... $@` for args
- `local` — always use inside functions to scope variables
- `arr=(a b c) · ${arr[@]} · ${#arr[@]}` — indexed array essentials
- `declare -A map` — associative arrays; `${!map[@]}` for keys

PROCESSES & JOBS

Every running program is a **process** with a unique **PID** (Process ID). The shell can run processes in the **foreground** (blocking the terminal) or **background** (appending `&`). The shell's own list of background processes are called **jobs**, managed with `jobs`, `fg`, and `bg`. Processes communicate via **signals** — numbered messages like `SIGTERM` (graceful stop) and `SIGKILL` (force kill). Scripts can intercept signals using `trap`.

PROCESS COMMANDS

```
# — Viewing processes —————
ps                # your processes in this terminal
ps aux            # all processes, all users (BSD style)
ps aux | grep python  # find python processes
pgrep python      # PIDs of processes named python
top               # interactive process viewer
htop              # better top (brew install htop)

# — Background & foreground —————
long_command &    # run in background; shell prompt returns
echo "Started PID: $!" # PID of background process
jobs              # list background jobs: [1] running...
fg                # bring last job to foreground
fg %1             # bring job 1 to foreground
bg %1             # resume stopped job in background
Ctrl+Z           # suspend foreground process (stop it)
Ctrl+C           # interrupt / kill foreground process

# — wait —————
cmd1 &
cmd2 &
wait              # wait for ALL background jobs to finish
wait $!           # wait for last background PID

# — Killing processes —————
kill 1234          # send SIGTERM (graceful stop) to PID 1234
kill -9 1234       # send SIGKILL (force kill — no cleanup)
kill -TERM 1234    # same as default kill
killall python     # kill all processes named python
pkill -f "script.sh" # kill by pattern match on full command

# — nohup & timeout —————
nohup ./script.sh & # keep running after logout; output → nohup.out
timeout 30 ./script.sh # kill script if it runs > 30 seconds
timeout 10s curl example.com # with explicit unit

# — trap — handle signals in scripts —————
trap "echo 'Caught SIGINT!'" SIGINT # intercept Ctrl+C
trap "rm -f /tmp/lockfile" EXIT     # cleanup on exit (always runs)
trap "echo interrupted; exit 1" INT TERM # handle multiple signals
trap - SIGINT                       # reset to default handler
```

COMMON MISTAKES

- Using `kill -9` immediately instead of `kill` — always try `SIGTERM` first; `SIGKILL` prevents proper cleanup
- Forgetting `wait` in scripts with background jobs — script exits before jobs finish, leaving orphan processes
- Not using `trap ... EXIT` for temp file cleanup — temp files accumulate if a script is interrupted
- Running long scripts without `nohup` over SSH — the script dies when the connection drops

TIPS

- Use `trap 'rm -f "$tmpfile"' EXIT` at the top of any script that creates temp files — automatic cleanup
- `pgrep -l python` shows name + PID — easier than parsing `ps aux | grep`
- For long-running SSH jobs, prefer `screen` or `tmux` over `nohup` — they let you reconnect and observe
- Use `timeout 30 cmd || echo "timed out"` to add time limits to any command in scripts

KEY PID VARIABLES

- `$$` — PID of the current shell / script
- `$!` — PID of the last background process
- `$?` — exit code of the last foreground command
- `$PPID` — PID of the parent process
- `$0` — name of the current script
- Exit 0 = success; any non-zero = error

NOTES

- `ps aux` — shows all users; `a` =all, `u` =user-oriented format, `x` =no tty required
- `&` — appended to any command; immediately returns shell prompt
- `Ctrl+Z` — suspends (pauses) the foreground job; use `bg` or `fg` to resume
- `kill -9` — `SIGKILL` cannot be caught or ignored; process is forcibly terminated by the OS
- `wait` — essential in scripts to synchronize parallel background jobs
- `trap ... EXIT` — runs cleanup code whether the script exits normally or via error
- `nohup` — redirects stdout to `nohup.out`; process ignores `SIGHUP` (terminal close)

COMMON SIGNALS

SIGNAL	NUMBER	MEANING
SIGHUP	1	Terminal closed / reload config
SIGINT	2	Interrupt (Ctrl+C)
SIGQUIT	3	Quit with core dump (Ctrl+\)
SIGKILL	9	Force kill — cannot be caught
SIGTERM	15	Graceful termination (default kill)
SIGSTOP	19	Pause process — cannot be caught
SIGCONT	18	Resume stopped process
SIGUSR1	30	User-defined signal 1
EXIT	—	Pseudo-signal: script exits (trap only)

JOB CONTROL QUICK REFERENCE

COMMAND / KEY	EFFECT
<code>cmd &</code>	Start in background
<code>Ctrl+C</code>	Kill foreground job
<code>Ctrl+Z</code>	Suspend foreground job
<code>jobs</code>	List all jobs and status
<code>fg %n</code>	Bring job <code>n</code> to foreground
<code>bg %n</code>	Resume job <code>n</code> in background
<code>kill %n</code>	Kill job <code>n</code> by job number
<code>wait</code>	Wait for all background jobs
<code>disown %n</code>	Detach job from shell (no SIGHUP)

SUMMARY

- `cmd &` runs in background · `jobs` lists · `fg` / `bg` move between foreground/background
- `kill PID` = `SIGTERM` (graceful) · `kill -9` = `SIGKILL` (force)
- `trap "cleanup" EXIT` — always runs on exit; use for temp file cleanup
- `wait` · `nohup` · `timeout` — essential for robust background scripts

TEXT PROCESSING

Unix text tools form a composable pipeline — each reads **stdin** and writes **stdout**, so they chain with `|` without temporary files. **sed** edits streams line by line using address ranges and `s/old/new/` substitutions; on macOS its `-i` flag requires an empty-string argument (`-i ''`). **awk** is a full micro-language: it splits each input line into fields (`$1...$NF`), supports BEGIN/END blocks, and can accumulate state across lines. **cut**, **sort**, **uniq**, **tr**, and **diff** handle field extraction, ordering, deduplication, character translation, and file comparison — pipe them together to transform any text without writing a dedicated script.

NOTES

- `sed -i ''` — macOS requires empty string arg with `-i ''`; Linux uses just `-i`
- `awk -F:` — sets the field separator; `$1` =first field, `$NF` =last field, `NR` =line number
- `awk END` — block runs after all lines are processed; use for totals
- `sort | uniq -c` — the classic frequency count pattern; sort results with `sort -rn`
- `tr` — works character by character; `-d` =delete, `-s` =squeeze repeated chars
- `column -t` — instant aligned table output; great for making CSV readable

SED · AWK · CUT

```
# — sed — stream editor —————
sed 's/old/new/' file.txt      # replace first match per line
sed 's/old/new/g' file.txt     # replace all matches (global)
sed 's/old/new/gI' file.txt    # case-insensitive replace
sed -i '' 's/foo/bar/g' file.txt # in-place edit (macOS: -i '' required)
sed -n '5,10p' file.txt        # print lines 5–10 only
sed '/^#/d' file.txt           # delete comment lines
sed 's/^[[:space:]]*/' file.txt # strip leading whitespace

# — awk — pattern / field processor —————
awk '{print $1}' file.txt      # print first field (whitespace=delim)
awk '{print $NF}' file.txt     # print last field
awk -F: '{print $1}' /etc/passwd # use : as delimiter
awk '{sum += $1} END {print sum}' # sum a column
awk 'NR==5' file.txt          # print line 5 only
awk 'NR>=5 && NR<=10' file.txt # print lines 5–10
awk '/error/ {print $0}' log.txt # lines matching pattern
awk '{print NR": "$0}' file.txt # number every line
awk -F, '{print $2, $4}' data.csv # CSV: print cols 2 and 4

# — cut — extract columns —————
cut -d: -f1 /etc/passwd       # field 1 with : delimiter
cut -d, -f2,4 data.csv        # fields 2 and 4 (CSV)
cut -c1-10 file.txt           # first 10 characters of each line
```

SORT · UNIQ · TR · DIFF

```
# — sort —————
sort file.txt                # alphabetical sort
sort -n numbers.txt          # numeric sort
sort -r file.txt             # reverse order
sort -u file.txt             # sort and remove duplicates
sort -t: -k3 -n /etc/passwd  # sort by field 3, : delim, numeric
sort -k2 -k1 file.txt        # sort by field 2, then field 1

# — uniq —————
sort file.txt | uniq         # remove consecutive duplicates
sort file.txt | uniq -c      # count occurrences
sort file.txt | uniq -d      # show only duplicated lines
sort file.txt | uniq -u      # show only unique lines

# — tr — translate / delete characters —————
tr 'a-z' 'A-Z' <<< "hello"  # HELLO (uppercase)
tr -d '\n' < file.txt       # remove all newlines
tr -s ' ' < file.txt        # squeeze repeated spaces to one
echo "a,b,c" | tr ', ' '\n' # comma → newline (split)

# — diff & column —————
diff file1.txt file2.txt     # show differences
diff -u file1.txt file2.txt  # unified format (git-style)
column -t -s: /etc/passwd    # align : delimited file into columns
column -t data.csv -s,       # pretty-print CSV
```

PRACTICAL ONE-LINERS

COMMAND	WHAT IT DOES
<code>sort file.txt uniq -c sort -rn head</code>	Top 10 most frequent lines
<code>awk '{sum+=\$1} END{print sum/NR}' file</code>	Average of numbers in column 1
<code>sed -n '/START/,/END/p' file.txt</code>	Print lines between two patterns
<code>grep -oE '\b[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}\b' -i file</code>	Extract email addresses
<code>awk '!seen[\$0]++' file.txt</code>	Remove duplicates preserving order (no sort)
<code>ps aux awk '{print \$1}' sort uniq -c sort -rn</code>	Count processes per user
<code>find . -name "*.js" xargs grep -l "TODO" sort</code>	JS files containing TODO, sorted
<code>cat file.txt tr ' ' '\n' sort uniq -c sort -rn</code>	Word frequency count
<code>diff <(ls dir1/) <(ls dir2/)</code>	Compare directory contents
<code>sed 's/[[:space:]]*/' file.txt</code>	Strip trailing whitespace from all lines

SED QUICK REFERENCE

SYNTAX	MEANING
<code>s/old/new/</code>	Substitute first match
<code>s/old/new/g</code>	Substitute all matches
<code>/pattern/d</code>	Delete matching lines
<code>/pattern/p</code>	Print matching lines (use <code>-n</code>)
<code>5,10p</code>	Print lines 5–10
<code>5,10d</code>	Delete lines 5–10
<code>\$d</code>	Delete last line
<code>1i\header</code>	Insert before line 1
<code>\$a\footer</code>	Append after last line

COMMON MISTAKES

- › Using `sed -i 'backup.bak'` on macOS — macOS `sed` requires a space: `sed -i .bak` or `sed -i ''`
- › Using `uniq` without `sort` first — `uniq` only removes adjacent duplicates; unsorted input gives wrong results
- › Forgetting `-F` in `awk` for non-space delimiters — default is any whitespace, not commas or colons

TIPS

- › Test `sed` without `-i` first to preview changes — add `-i ''` only when the output looks right
- › `awk '!seen[$0]++'` is the idiomatic way to deduplicate while preserving order — memorize it
- › Combine `column -t` with `less -S` to browse wide tabular output without line wrapping
- › Use `sd` (brew install sd) as a modern `sed` alternative with cleaner syntax for quick substitutions

SUMMARY

- › `sed` — line-by-line substitution and deletion; `-i ''` for in-place on macOS
- › `awk` — field-based processing; `$1 ... $NF`, `NR`, `BEGIN` / `END`
- › `sort | uniq -c | sort -rn` — the frequency count pipeline
- › `cut` · `tr` · `column` — quick field extraction, char translation, table formatting

ROBUST SCRIPT TEMPLATE

```
#!/usr/bin/env bash
set -euo pipefail          # exit on error, unbound vars, pipe fail
IFS=$'\n\t'               # safer word splitting

# — Script metadata —————
readonly SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
readonly SCRIPT_NAME="$(basename "$0")"
readonly LOG_FILE="/tmp/${SCRIPT_NAME}.log"

# — Logging helpers —————
log() { echo "[$(date +%H:%M:%S)] $*" | tee -a "$LOG_FILE"; }
info() { log "INFO $*"; }
warn() { log "WARN $*" >&2; }
error() { log "ERROR $*" >&2; exit 1; }

# — Cleanup on exit —————
tmpfile=$(mktemp)
trap 'rm -f "$tmpfile"' EXIT

# — Argument parsing with getopts —————
usage() { echo "Usage: $0 [-v] [-o output] input"; exit 1; }
verbose=false; output=""
while getopts ":vo:h" opt; do
  case $opt in
    v) verbose=true ;;
    o) output="$OPTARG" ;;
    h) usage ;;
    :) error "Option -$OPTARG requires argument" ;;
    \?) error "Unknown option: -$OPTARG" ;;
  esac
done
shift $(( OPTIND - 1 ))      # remaining args start at $1

# — Require argument check —————
[[ $# -lt 1 ]] && usage
input="${1:?Input file required}"

# — Color output —————
RED='\033[0;31m'; GREEN='\033[0;32m'
YELLOW='\033[1;33m'; NC='\033[0m' # NC = No Color
echo -e "${GREEN}Success${NC}"
echo -e "${RED}Error${NC}"
echo -e "${YELLOW}Warning${NC}"

# — Debugging —————
set -x                    # print each command before executing
set +x                   # stop tracing
bash -n script.sh        # syntax check without running
bash -x script.sh        # run with trace output
```

NOTES

- `set -e` — exit immediately on any error; prevents cascading failures
- `set -u` — treat unset variables as errors; catches typos in variable names
- `set -o pipefail` — pipeline fails if any command in it fails (not just the last)
- `${BASH_SOURCE[0]}` — reliable way to get the script's own path, even when sourced
- `getopts` — POSIX standard arg parser; handles `-v` style flags; use `OPTARG` for values
- `trap ... EXIT` — cleanup always runs whether exit is normal, error, or signal
- `bash -n` — syntax check only; essential before deploying a script

HOMEBREW ESSENTIALS

COMMAND	DESCRIPTION
<code>brew install pkg</code>	Install a package
<code>brew uninstall pkg</code>	Remove a package
<code>brew upgrade</code>	Upgrade all installed packages
<code>brew upgrade pkg</code>	Upgrade one package
<code>brew update</code>	Update Homebrew itself
<code>brew list</code>	List installed packages
<code>brew search term</code>	Search available packages
<code>brew info pkg</code>	Package details and deps
<code>brew doctor</code>	Diagnose installation issues
<code>brew cleanup</code>	Remove old versions

SCRIPT BEST PRACTICES

PRACTICE	WHY
<code>set -euo pipefail</code>	Fail fast; catch all errors
Quote all variables	Prevents word-splitting bugs
<code>trap ... EXIT</code>	Guaranteed cleanup code
<code>readonly constants</code>	Prevent accidental mutation
<code>local in functions</code>	Prevent scope leaks
<code>bash -n</code> before deploy	Catch syntax errors early
Use <code>mktemp</code> for temp files	Unique, safe temp paths
<code>[[]]</code> not <code>[]</code>	Safer, more features
Print usage on <code>-h</code>	Self-documenting scripts

MACOS-SPECIFIC TERMINAL TOOLS

COMMAND	DESCRIPTION	EXAMPLE
<code>pbcopy</code>	Copy stdin to clipboard	<code>cat file.txt pbcopy</code>
<code>pbpaste</code>	Paste clipboard to stdout	<code>pbpaste > newfile.txt</code>
<code>open</code>	Open file/URL in default app	<code>open index.html</code>
<code>open -a</code>	Open with specific app	<code>open -a Xcode project.xcodeproj</code>
<code>open -R</code>	Reveal file in Finder	<code>open -R ./build/app.dmg</code>
<code>say</code>	Text-to-speech	<code>say "Build complete"</code>
<code>osascript</code>	Run AppleScript / JavaScript	<code>osascript -e 'display dialog "Done"'</code>
<code>defaults</code>	Read/write macOS preferences	<code>defaults write com.apple.dock autohide -bool true</code>
<code>caffeinate</code>	Prevent sleep while running	<code>caffeinate -i ./long-script.sh</code>
<code>networksetup</code>	Configure network from CLI	<code>networksetup -listallnetworkservices</code>
<code>diskutil</code>	Disk management CLI	<code>diskutil list</code>
<code>launchctl</code>	Manage launch agents/daemons	<code>launchctl list grep my.agent</code>
<code>mdfind</code>	Spotlight search from CLI	<code>mdfind "kMDItemKind == 'PDF'"</code>
<code>sw_vers</code>	macOS version info	<code>sw_vers -productVersion</code>

COMMON MISTAKES

- › Omitting `set -euo pipefail` — scripts silently continue past errors, leaving things in broken state
- › Using `${SCRIPT_DIR}` as just `${dirname $0}` — breaks when script is called from another directory; use `${BASH_SOURCE[0]}`
- › Hardcoding paths like `/usr/local/bin` — Apple Silicon Macs use `/opt/homebrew/bin`; use `command -v` instead

TIPS

- › Add `say "Script complete"` at the end of long-running scripts — get an audio notification when done
- › Use `cmd | pbcopy` constantly — pipe any output directly to clipboard for pasting elsewhere
- › Use `osascript -e 'display notification "Done" with title "Script"'` for macOS notifications from scripts
- › Use `shellcheck` (brew install shellcheck) to lint scripts — catches dozens of common bash mistakes automatically

SUMMARY

- › `set -euo pipefail` — the single most important line to add to every script
- › `getopts` · `trap EXIT` · `mktemp` — the three pillars of robust scripts
- › `pbcopy` · `open` · `say` · `osascript` — macOS power tools for automation
- › `shellcheck` + `bash -n` — lint and syntax-check before every deploy